

Experiment 'p_6_2_d_replace_with_larger_neighbor_array'

Results

June 4, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing assume : 0
Responses containing expect : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_6_2_d_replace_with_larger_neighbor_array

Natural language statement: Write a method to replace each element of an array, except the first and last, by the larger of its two neighbors.

Method signature: p_6_2_d_replace_with_larger_neighbor_array(inputs: array<int>) returns (result: array<int>)

Ensures

- result.Length == inputs.Length
- result[0] == inputs[0]
- result[inputs.Length - 1] == inputs[inputs.Length - 1]
- forall i :: 1 <= i < inputs.Length - 1 ==> result[i] == if inputs[i - 1] >= inputs[i + 1] then inputs[i - 1] else inputs[i + 1]

Requires

- inputs.Length >= 3

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to replace each element of an array, except the first and last, by the
↪ larger of its two neighbors.

The signature should be:

p_6_2_d_replace_with_larger_neighbor_array(inputs: array<int>) returns (result: array<int>
↪ >)

The method should respect the following contract:

requires inputs.Length >= 3, ensures result.Length == inputs.Length, ensures result[0] ==
↪ inputs[0], ensures result[inputs.Length - 1] == inputs[inputs.Length - 1],
↪ ensures forall i :: 1 <= i < inputs.Length - 1 ==> result[i] == if inputs[i - 1]
↪ >= inputs[i + 1] then inputs[i - 1] else inputs[i + 1]

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn0GhbJ3ywaU7WLf8g7cf1JzcpWVM

Model: gpt-5.5-2026-04-23

Created at: 1780571475

Finish reason: stop

Usage: 454 tokens in, and 747 tokens out

```
{
  result := new int[inputs.Length];
  var n := inputs.Length;

  result[0] := inputs[0];
  result[n - 1] := inputs[n - 1];

  var i := 1;
  while i < n - 1
    invariant n == inputs.Length
    invariant result.Length == n
    invariant 1 <= i <= n - 1
    invariant result[0] == inputs[0]
    invariant result[n - 1] == inputs[n - 1]
    invariant forall j :: 1 <= j < i ==> result[j] == if inputs[j - 1] >= inputs[j + 1]
      ↪ then inputs[j - 1] else inputs[j + 1]
```

```

{
  result[i] := if inputs[i - 1] >= inputs[i + 1] then inputs[i - 1] else inputs[i + 1];
  i := i + 1;
}
}

```

Final Program

```

method p_6_2_d_replace_with_larger_neighbor_array(inputs: array<int>) returns (result:
  ↪ array<int>)
  requires inputs.Length >= 3
  ensures result.Length == inputs.Length
  ensures result[0] == inputs[0]
  ensures result[inputs.Length - 1] == inputs[inputs.Length - 1]
  ensures forall i :: 1 <= i < inputs.Length - 1 ==> result[i] == if inputs[i - 1]
    ↪ >= inputs[i + 1] then inputs[i - 1] else inputs[i + 1]
{
  result := new int[inputs.Length];
  var n := inputs.Length;

  result[0] := inputs[0];
  result[n - 1] := inputs[n - 1];

  var i := 1;
  while i < n - 1
    invariant n == inputs.Length
    invariant result.Length == n
    invariant 1 <= i <= n - 1
    invariant result[0] == inputs[0]
    invariant result[n - 1] == inputs[n - 1]
    invariant forall j :: 1 <= j < i ==> result[j] == if inputs[j - 1] >= inputs[j + 1]
      ↪ then inputs[j - 1] else inputs[j + 1]
    {
      result[i] := if inputs[i - 1] >= inputs[i + 1] then inputs[i - 1] else inputs[i + 1];
      i := i + 1;
    }
  }
}

```

Total Token Usage

Input tokens: 454

Output tokens: 747

Reasoning tokens: 512

Sum of 'total tokens': 1201

Experiment Timings

Overall Experiment started at 1780571474551, ended at 1780571518673, lasting 44122ms (44.12 seconds)

Iteration #1 started at 1780571474553, ended at 1780571518673, lasting 44120ms (44.12 seconds)